

train-test-split

- (1) from sklearn.model_selection import train_test_split
x_train, y_train, x_test, y_test = train_test_split(x, y, test_size = 0.2,
random_state = 42)
- (2) from sklearn.model_selection import train_test_split
x_train, y_train, x_test, y_test = train_test_split(x, y, test_size = 0.2,
random_state = 42)
- (3) from sklearn.model_selection import train_test_split
x_train, y_train, x_test, y_test = train_test_split(x, y, test_size = 0.2,
random_state = 42)
- (4) from sklearn.model_selection import train_test_split
x_train, y_train, x_test, y_test = train_test_split(x, y, test_size = 0.2,
random_state = 42)
- (5) from sklearn.model_selection import train_test_split
x_train, y_train, x_test, y_test = train_test_split(x, y, test_size = 0.2,
random_state = 42)

Regression

Linear

- (1) from sklearn.linear_model import LinearRegression
model = LinearRegression()
model.fit(x_train, y_train)
y_pred = model.predict(x_test)
- (2) from sklearn.linear_model import LinearRegression
model = LinearRegression()
model.fit(x_train, y_train)
y_pred = model.predict(x_test)

- (3) from sklearn.linear_model import LinearRegression
model = LinearRegression()
model.fit(x_train, y_train)
y_pred = model.predict(x_test)
- (4) from sklearn.linear_model import LinearRegression
model = LinearRegression()
model.fit(x_train, y_train)
y_pred = model.predict(x_test)
- (5) from sklearn.linear_model import LinearRegression
model = LinearRegression()
model.fit(x_train, y_train)
y_pred = model.predict(x_test)

ridge

- (1) from sklearn.linear_model import Ridge
model = Ridge(alpha=1.0)
model.fit(x_train, y_train)
- (2) from sklearn.linear_model import Ridge
model = Ridge(alpha=1.0)
model.fit(x_train, y_train)
- (3) from sklearn.linear_model import Ridge
model = Ridge(alpha=1.0)
model.fit(x_train, y_train)
- (4) from sklearn.linear_model import Ridge
model = Ridge(alpha=1.0)
model.fit(x_train, y_train)
- (5) from sklearn.linear_model import Ridge
model = Ridge(alpha=1.0)
model.fit(x_train, y_train)

Lasso

- (1)

```
from sklearn.linear_model import Lasso  
model = Lasso(alpha = 0.1)  
model.fit(x_train, y_train)
```
- (2)

```
from sklearn.linear_model import Lasso  
model = Lasso(alpha = 0.1)  
model.fit(x_train, y_train)
```
- (3)

```
from sklearn.linear_model import Lasso  
model = Lasso(alpha = 0.1)  
model.fit(x_train, y_train)
```
- (4)

```
from sklearn.linear_model import Lasso  
model = Lasso(alpha = 0.1)  
model.fit(x_train, y_train)
```
- (5)

```
from sklearn.linear_model import Lasso  
model = Lasso(alpha = 0.1)  
model.fit(x_train, y_train)
```

Polynomial

- (1)

```
from sklearn.preprocessing import PolynomialFeatures  
model = PolynomialFeatures(degree = 3)  
model.fit_transform(x)
```
- (2)

```
from sklearn.preprocessing import PolynomialFeatures  
model = PolynomialFeatures(degree = 3)  
model.fit_transform(x)
```
- (3)

```
from sklearn.preprocessing import PolynomialFeatures  
model = PolynomialFeatures(degree = 3)  
model.fit_transform(x)
```
- (4)

```
from sklearn.preprocessing import PolynomialFeatures  
model = PolynomialFeatures(degree = 3)  
model.fit_transform(x)
```
- (5)

```
from sklearn.preprocessing import PolynomialFeatures  
model = PolynomialFeatures(degree = 3)  
model.fit_transform(x)
```


Classification

Logistic regression

- (1)

```
from sklearn.linear_model import LogisticRegression  
model = LogisticRegression()  
model.fit(x_train, y_train)
```
- (2)

```
from sklearn.linear_model import LogisticRegression  
model = LogisticRegression()  
model.fit(x_train, y_train)
```
- (3)

```
from sklearn.linear_model import LogisticRegression  
model = LogisticRegression()  
model.fit(x_train, y_train)
```
- (4)

```
from sklearn.linear_model import LogisticRegression  
model = LogisticRegression()  
model.fit(x_train, y_train)
```
- (5)

```
from sklearn.linear_model import LogisticRegression  
model = LogisticRegression()  
model.fit(x_train, y_train)
```

k-nearest neighbors

- (1)

```
from sklearn.neighbors import KNeighborsClassifier  
model = KNeighborsClassifier(n_neighbors=5)  
model.fit(x_train, y_train)
```
- (2)

```
from sklearn.neighbors import KNeighborsClassifier  
model = KNeighborsClassifier(n_neighbors=5)  
model.fit(x_train, y_train)
```
- (3)

```
from sklearn.neighbors import KNeighborsClassifier  
model = KNeighborsClassifier(n_neighbors=5)  
model.fit(x_train, y_train)
```

(4) from sklearn.neighbors import KNeighborsClassifier
model = KNeighborsClassifier (n_neighbors=5)
model.fit (x_train, y_train)

(5) from sklearn.neighbors import KNeighborsClassifier
model = KNeighborsClassifier (n_neighbors=5)
model.fit (x_train, y_train)

support vector machine

(1) from sklearn.svm import SVC
model = SVC (kernel = 'rbf')
model.fit (x_train, y_train)

(2) from sklearn.svm import SVC
model = SVC (kernel = 'rbf')
model.fit (x_train, y_train)

(3) from sklearn.svm import SVC
model = SVC (kernel = 'rbf')
model.fit (x_train, y_train)

(4) from sklearn.svm import SVC
model = SVC (kernel = 'rbf')
model.fit (x_train, y_train)

(5) from sklearn.svm import SVC
model = SVC (kernel = 'rbf')
model.fit (x_train, y_train)

decision tree classifier

(1) from sklearn.tree import DecisionTreeClassifier
model = DecisionTreeClassifier()
model.fit(x_train, y_train)

(2) from sklearn.tree import DecisionTreeClassifier
model = DecisionTreeClassifier()
model.fit(x_train, y_train)

(3) from sklearn.tree import DecisionTreeClassifier
model = DecisionTreeClassifier()
model.fit(x_train, y_train)

(4) from sklearn.tree import DecisionTreeClassifier
model = DecisionTreeClassifier()
model.fit(x_train, y_train)

(5) from sklearn.tree import DecisionTreeClassifier
model = DecisionTreeClassifier()
model.fit(x_train, y_train)

Random forest classifier

(1) from sklearn.ensemble import RandomForestClassifier
model = RandomForestClassifier()
model.fit(x_train, y_train)

(2) from sklearn.ensemble import RandomForestClassifier
model = RandomForestClassifier()
model.fit(x_train, y_train)

(3) from sklearn.ensemble import RandomForestClassifier
model = RandomForestClassifier()
model.fit(x_train, y_train)

(1) from sklearn.ensemble import RandomForestClassifier
model = RandomForestClassifier()
model.fit(x_train, y_train)

RandomForest Classifier

(2) from sklearn.ensemble import RandomForestClassifier
model = RandomForestClassifier()
model.fit(x_train, y_train)

RandomForest Classifier

* Naive Bayes

(1) from sklearn.naive-bayes import GaussianNB
model = GaussianNB()
model.fit(x_train, y_train)

GaussianNB

(2) from sklearn.naive-bayes import GaussianNB
model = GaussianNB()
model.fit(x_train, y_train)

GaussianNB

(3) from sklearn.naive-bayes import GaussianNB
model = GaussianNB()
model.fit(x_train, y_train)

GaussianNB

(4) from sklearn.naive-bayes import GaussianNB
model = GaussianNB()
model.fit(x_train, y_train)

GaussianNB

(5) from sklearn.naive-bayes import GaussianNB
model = GaussianNB()
model.fit(x_train, y_train)

GaussianNB

Clustering

k-means

- (1) from sklearn.cluster import KMeans
model = KMeans (n_clusters = 3)
model.fit(x)
labels = model.predict(x) KMeans
- (2) from sklearn.cluster import KMeans
model = KMeans (n_clusters = 3)
model.fit(x)
labels = model.predict(x) KMeans
- (3) from sklearn.cluster import KMeans
model = KMeans (n_clusters = 3)
model.fit(x)
labels = model.predict(x) KMeans
- (4) from sklearn.cluster import KMeans
model = KMeans (n_clusters = 3)
model.fit(x)
labels = model.predict(x) KMeans
- (5) from sklearn.cluster import KMeans
model = KMeans (n_clusters = 3)
model.fit(x)
labels = model.predict(x) KMeans

dbscan

- (1) from sklearn.cluster import DBSCAN
model = DBSCAN (eps = 0.5, min_samples = 5)
labels = model.predict(x)
- (2) from sklearn.cluster import DBSCAN
model = DBSCAN (eps = 0.5, min_samples = 5)
labels = model.predict(x)

(3) from sklearn.cluster import DBSCAN
model = DBSCAN (eps = 0.5, min_samples = 5)
labels = model.predict(x)

(4) from sklearn.cluster import DBSCAN
model = DBSCAN (eps = 0.5, min_samples = 5)
labels = model.predict(x)

(5) from sklearn.cluster import DBSCAN
model = DBSCAN (eps = 0.5, min_samples = 5)
labels = model.predict(x)

gmm (Gaussian mixture models)

(1) from ~~sklearn.multiple~~

(1) from sklearn.mixture import GaussianMixture
model = GaussianMixture (n_components = 3)
model.fit(x)
labels = model.predict(x)

(2) from sklearn.mixture import GaussianMixture
model = GaussianMixture (n_components = 3)
model.fit(x)
labels = model.predict(x)

(3) from sklearn.mixture import GaussianMixture
model = GaussianMixture (n_components = 3)
~~model~~.fit(x)
labels = model.predict(x)

(4) from sklearn.mixture import GaussianMixture
model = GaussianMixture (n_components = 3)
model.fit(x)
labels = model.predict(x)

```
(5) from sklearn.mixture import GaussianMixture  
model = GaussianMixture(n_components=3)  
model.fit(x)  
labels = model.predict(x)
```

Hierarchical

```
(1) from sklearn.cluster import AgglomerativeClustering  
model = AgglomerativeClustering()  
labels = model.fit_predict(x)
```

```
(2) from sklearn.cluster import AgglomerativeClustering  
model = AgglomerativeClustering()  
labels = model.fit_predict(x)
```

```
(3) from sklearn.cluster import AgglomerativeClustering  
model = AgglomerativeClustering()  
labels = model.fit_predict(x)
```

```
(4) from sklearn.cluster import AgglomerativeClustering  
model = AgglomerativeClustering()  
labels = model.fit_predict(x)
```

```
(5) from sklearn.cluster import AgglomerativeClustering  
model = AgglomerativeClustering()  
labels = model.fit_predict(x)
```

Association rule learning:

pip install mlxtend

- (1)

```
from mlxtend.frequent_patterns import apriori, association_rules
```
- (2)

```
from mlxtend.frequent_patterns import fpgrowth, association_rules
```
- (3)

```
from mlxtend.frequent_patterns import apriori, association_rules
```

```
from mlxtend.frequent_patterns import fpgrowth, association_rules
```
- (4)

```
from mlxtend.frequent_patterns import apriori, association_rules
```

```
from mlxtend.frequent_patterns import fpgrowth, association_rules
```
- (5)

```
from mlxtend.frequent_patterns import apriori, association_rules
```

```
from mlxtend.frequent_patterns import fpgrowth, association_rules
```